

Tipos Algébricos

João Eduardo Montandon
DCC024

Introdução

Um tipo primitivo é qualquer tipo que pode ser utilizado, mas não definido no programa.

-- [Aula: Tipos](#)

Quais tipos são primitivos em ML? 🤖

- `int` ?
 - `real` ?
 - `char` ?
 - `bool` ?
-

Um `datatype` representa uma união de tipos, onde cada elemento da união é rotulado.

Cada elemento pode ser tão simples quanto um nome (`true`) ou complicado como uma função.

```
datatype bool = True | False;
```

Terminologia Básica

- `bool` : Construtor de tipos
 - `True` , `False` : Construtor de dados
-

Enumerações

- A forma mais simples de se representar datatypes em ML

- Cada elemento é um possível valor da enumeração

```
datatype dia = Seg | Ter | Qua | Qui | Sex | Sab | Dom;  
fun ehFds d = (d = Sab orelse d = Dom);  
  
ehFds Dom;  
ehFds Ter;
```

```
(* Exemplo com pattern matching *)  
fun ehFds Sab = true  
|   ehFds Dom = true  
|   ehFds _ = false;  
  
ehFds Dom;  
ehFds Ter;
```

Construtores De Dados Com Parâmetros

Parâmetros ➡ utilizados para **produzirem novos elementos**.

```
datatype Z = Valor of int | MaisInfinito | MenosInfinito;  
MaisInfinito;  
MenosInfinito;  
Valor;  
Valor 20;
```

O que acontece se eu executar isso? 🤔

```
val x = Valor 5;  
x + x;
```

Qual o tipo de `x` ?

Para operar o valor contido em `x`, é necessário fazer o *unwrap* do valor.

```
- val (Valor xInt) = x;  
val xInt = 5 : int  
- xInt + xInt;  
val it = 10 : int
```

Obs? Valor não é uma função!

Exemplo: Função para calcular o quadrado de Z

```
fun quadradoZ n = case n of  
    MenosInfinito => MaisInfinito |  
    MaisInfinito => MaisInfinito |  
    Valor nInt => Valor(nInt * nInt);
```

```
- val x1 = quadradoZ x;  
val x1 = Valor 25 : Z  
- quadradoZ (Valor 10000000000);  
uncaught exception Overflow [overflow]
```

Plus: Tratando números muito grandes

```
fun quadradoZ n = case n of  
    MenosInfinito => MaisInfinito |  
    MaisInfinito => MaisInfinito |  
    Valor nInt => Valor(nInt * nInt)  
        handle Overflow => MaisInfinito;
```

```
- quadradoZ (Valor 100000000000);  
val it = MaisInfinito :
```

Construtores De Tipos Com Parâmetros

Parâmetros → utilizados para [definir o tipo](#) dos dados.

Exemplo: Option ^[1]

```
datatype 'a option = NONE | SOME of 'a;
```

- `option` recebeu o tipo `'a` como parâmetro.
- `'a` é o tipo do valor a ser recebido por `SOME`.

```
- SOME 4;  
val it = SOME 4 : int option  
- SOME "frase"  
val it = SOME "frase" : string option  
- NONE;  
val it = NONE : 'a option
```

Para quê serve? 🤔

```
fun divSegura n 0 = NONE  
|   divSegura n d = SOME(n div d);  
  
- meuDiv 4 2;  
val it = SOME 2 : int option  
- meuDiv 4 0;  
val it = NONE : int option
```

Exemplo: punhado

```
datatype 'coisas punhado = Um of 'coisas | Varios of 'coisas list;  
  
- Um 10.0;  
val it = Um 10.0 : real punhado  
- Varios [1,2,3];  
val it = Varias [1,2,3] : int punhado
```

Podemos tirar proveito do polimorfismo existente para computar punhados de diferentes tipos da mesma forma

```
fun tamanho (Um _) = 1  
|   tamanho (Varios l) = length l;
```

```
tamanho (Um 1);  
tamanho (Um "haha");  
tamanho (Varios [1,2,3,4,5,6]);  
tamanho (Varios ["a", "b", "c"]);  
  
tamanho (Um [1,2,3,4,5]); (* Qual o valor dessa execução? *)
```

Funciona? Pq sim? Pq não? 🤔

```
datatype int valor = NADA | ALGUM of int;
```

datatype Recursivos

É possível definir um `datatype` em termos de si mesmo.

Onde isso é utilizado? 🤔

Tipo definido em termos de si mesmo em C

```
struct Node {  
    int val;  
    Node* next;  
}
```

Tipo definido em termos de si mesmo em ML

```
datatype intList = NIL | NODE of int * intList;
```

```
- NIL;  
val it = NIL : intList  
- NODE;  
val it = fn : int * intList -> intList  
- NODE(1, NIL);
```

```
val it = NODE (1,NIL) : intList
- NODE(1, NODE(2, NODE(3, NIL)));
val it = NODE (1,NODE (2,NODE (3,NIL))) : intList
```

Soa familiar? 🤔

Podemos parametrizar o tipo do `datatype` para deixá-lo genérico.

```
datatype 'a myList = NIL | NODE of 'a * 'a myList;

- NIL;
val it = NIL : 'a myList
- NODE;
val it = fn : 'a * 'a myList -> 'a myList
- NODE(1, NODE(2, NODE(3, NIL)));
val it = NODE (1,NODE (2,NODE (3,NIL))) : int myList
```

Qual a diferença entre eles? 🤔

```
- NODE;
val it = fn : 'a * 'a myList -> 'a myList
- op ::;
val it = fn : 'a * 'a list -> 'a list
```

Veja as referências para mais.^[2]

Se ambos tipos possuem expressividade equivalente...

```
datatype 'a myList = NIL | || of 'a * 'a myList; 🤔
infixr 5 ||;

- val minhaLista = 1 || 2 || 3 || 4 || 5 || NIL; 🤔
val minhaLista = 1 || 2 || 3 || 4 || 5 || NIL : int myList 😎
```

Exemplo: myLength

```
fun mylength NIL = 0
|   mylength (_ || l) = 1 + mylength l;

mylength minhaLista;
```

Exemplo: MyMap

```
fun mymap f NIL = NIL
|   mymap f (h || t) = f(h) || mymap f t;

- mymap (~) minhaLista;
val it = ~1 || ~2 || ~3 || ~4 || ~5 || NIL : int myList
```

Exemplo: MyFoldr

```
fun myfoldr f c NIL = c
|   myfoldr f c (a || b) = f(a, myfoldr f c b);

myfoldr (op +) 0 minhaLista;
```

Considerações Finais

- Nossa jornada rumo ao paradigma funcional fica por aqui.
- Programação funcional se reduz a

$$D \xrightarrow{f_{un}} I$$

- Onde uma implementação de f deve existir para cada Combinação entre D e I .
-

- Apenas uma introdução a ML, há varias outras coisas para se ver
 - Records
 - Arrays
 - Encapsulamento
 - APIs
-

Lembre-se Do Principal

- Estudamos um estilo de programação orientado a funções
 - É uma outra perspectiva de se resolver problemas, **não a única!**
-
-

1. [The Option structure](#) ↩

2. [The List structure](#) ↩